

Pack 30 (API + Web) — Scheduled Key Rotation with Overlap Windows + Rotation Policy (real code)

RemoteDesk · Pack 30 (API + Web) — Scheduled Key Rotation with Overlap Windows + Rotation Policy (real code)

Codex / Claude Code handoff. Direct continuation of Packs 26 to 29. Pack 27 added MANUAL key rotation; this slice makes it AUTOMATIC and safe: a configurable rotation policy (max key age + overlap window), a scheduled rotation runner that mints a new active key before retiring the old one so there's no signing gap, and a grace window during which a just-retired key is still flagged as overlapping.

Scope: rotate signing keys on a schedule without breaking in-flight signing or historical verification. **Overlap window guarantees a valid active key at all times; retired keys verify forever (never deleted). Runner is idempotent. Reuses Pack 27 rotate + Pack 25 audit. No private key handling beyond what 27 already isolates.**

Tag legend: **[NEW]** = create file. **[MODIFY]** = patch existing file by hand (merge, don't overwrite).

Files created / modified / not touched

CREATED [NEW]	
packages/shared/src/permissions/rotationPolicy.ts	
apps/api/prisma/migrations/0030_rotation_policy/migration.sql	
apps/api/src/repositories/rotationPolicyRepository.ts	
apps/api/src/services/admin/scheduledRotation.service.ts	
apps/api/src/jobs/rotationRunner.ts	
apps/web/app/dashboard/receipts/RotationPolicyPanel.tsx	
tests/admin/rotationPolicy.test.ts	
tests/admin/scheduledRotation.service.test.ts	
MODIFIED [MODIFY]	
packages/shared/src/permissions/index.ts	(export rotationPolicy)
apps/api/prisma/schema.prisma	(KeyRotationPolicy model)
apps/api/src/routes/deviceAdmin.routes.ts	(get/set rotation policy; run-now)
apps/web/app/dashboard/receipts/page.tsx	(mount RotationPolicyPanel)
NOT TOUCHED (intentionally)	
apps/desktop/**	(verification already pin-aware in Pack 28)
apps/api/src/lib/kms/**	(envelope crypto unchanged)
apps/api/src/services/admin/receiptSigner.service.ts	(still signs with the active key)

Part A — Shared: rotation policy contract + evaluator

FILE: packages/shared/src/permissions/rotationPolicy.ts

```

import type { SigningPublicKey } from './receiptSignature';

/**
 * Rotation policy: how often to rotate the receipt signing key and how long a
 * just-retired key remains flagged as overlapping. Pure helpers decide when
 * rotation is due and whether a retired key is still within grace.
 *
 * Overlap semantics: a new active key is minted BEFORE the old one is retired, so
 * there is never a moment without a valid signing key. After retirement, the old
 * key still VERIFIES forever (never deleted) but is flagged "in grace" for a
 * window so clients/auditors can treat a recent rotation leniently.
 */
export interface RotationPolicy {
  maxKeyAgeDays: number;
  overlapDays: number;
  enabled: boolean;
}

export const DEFAULT_ROTATION_POLICY: RotationPolicy = {
  maxKeyAgeDays: 90,
  overlapDays: 7,
  enabled: false, // opt-in: rotation is off until an admin enables it
};

/** Clamp + validate a policy from untrusted input. */
export function normalizeRotationPolicy(p: Partial<RotationPolicy>): RotationPolicy {
  return {
    maxKeyAgeDays: clampInt(p.maxKeyAgeDays ?? DEFAULT_ROTATION_POLICY.maxKeyAgeDays, 1, 3650),
    overlapDays: clampInt(p.overlapDays ?? DEFAULT_ROTATION_POLICY.overlapDays, 0, 365),
    enabled: !!p.enabled,
  };
}

function clampInt(n: number, min: number, max: number): number {
  const v = Math.floor(Number.isFinite(n) ? n : min);
  return Math.min(max, Math.max(min, v));
}

/** Age of the active key in whole days. */
export function keyAgeDays(activeKey: SigningPublicKey, now: Date = new Date()): number {
  return Math.floor((now.getTime() - Date.parse(activeKey.createdAt)) / 86_400_000);
}

/**
 * Is rotation due? True only when enabled, an active key exists, and its age
 * exceeds maxKeyAgeDays. Fail-safe: no active key => due (so a key gets minted).
 */
export function isRotationDue(policy: RotationPolicy, activeKey: SigningPublicKey | null, now: Date = new Date()):
boolean {
  if (!policy.enabled) return false;
  if (!activeKey) return true;
  return keyAgeDays(activeKey, now) >= policy.maxKeyAgeDays;
}

/** Is a retired key still within its overlap/grace window? */
export function isWithinOverlap(policy: RotationPolicy, retiredKey: Pick<SigningPublicKey, 'retiredAt'>, now: Date = new

```

```
Date()): boolean {
  if (!retiredKey.retiredAt) return false;
  const graceEnd = Date.parse(retiredKey.retiredAt) + policy.overlapDays * 86_400_000;
  return now.getTime() < graceEnd;
}
```

FILE: packages/shared/src/permissions/index.ts [MODIFY]

```
// ...existing exports through Pack 29...
export * from './rotationPolicy'; // ADD
```

Part B — API: Prisma schema + migration

FILE: apps/api/prisma/schema.prisma [MODIFY]

```
// ---- ADD: per-org (or global) rotation policy ----
model KeyRotationPolicy {
  id          String   @id @default(cuid())
  orgId       String?  @unique // null = workspace-global policy
  maxKeyAgeDays Int     @default(90)
  overlapDays Int      @default(7)
  enabled     Boolean  @default(false)
  lastRunAt   DateTime?
  updatedAt   DateTime @updatedAt
  createdAt   DateTime @default(now())
}
```

FILE: apps/api/prisma/migrations/0030_rotation_policy/migration.sql

```
CREATE TABLE "KeyRotationPolicy" (
  "id"          TEXT PRIMARY KEY,
  "orgId"       TEXT UNIQUE,
  "maxKeyAgeDays" INTEGER NOT NULL DEFAULT 90,
  "overlapDays" INTEGER NOT NULL DEFAULT 7,
  "enabled"     BOOLEAN NOT NULL DEFAULT false,
  "lastRunAt"   TIMESTAMP,
  "updatedAt"   TIMESTAMP NOT NULL DEFAULT now(),
  "createdAt"   TIMESTAMP NOT NULL DEFAULT now()
);
CREATE INDEX "KeyRotationPolicy_orgId_idx" ON "KeyRotationPolicy"("orgId");
```

Part C — API: policy repo + scheduled rotation service

FILE: apps/api/src/repositories/rotationPolicyRepository.ts

```
import { normalizeRotationPolicy, DEFAULT_ROTATION_POLICY, type RotationPolicy } from '@remotedesk/shared';
import { prisma } from '../db/prisma';

export const rotationPolicyRepository = {
  async get(orgId: string): Promise<RotationPolicy & { lastRunAt: string | null }> {
    const row = await prisma.keyRotationPolicy.findUnique({ where: { orgId } });
    if (!row) return { ...DEFAULT_ROTATION_POLICY, lastRunAt: null };
    return {
```

```

        maxKeyAgeDays: row.maxKeyAgeDays,
        overlapDays: row.overlapDays,
        enabled: row.enabled,
        lastRunAt: row.lastRunAt ? row.lastRunAt.toISOString() : null,
    };
},

async set(orgId: string, input: Partial<RotationPolicy>): Promise<RotationPolicy> {
    const policy = normalizeRotationPolicy(input);
    await prisma.keyRotationPolicy.upsert({ where: { orgId }, create: { orgId, ...policy }, update: policy });
    return policy;
},

async markRun(orgId: string, at: Date = new Date()): Promise<void> {
    await prisma.keyRotationPolicy.updateMany({ where: { orgId }, data: { lastRunAt: at } });
},

async allEnabled(): Promise<Array<{ orgId: string; policy: RotationPolicy }>> {
    const rows = await prisma.keyRotationPolicy.findMany({ where: { enabled: true, orgId: { not: null } } });
    return rows.map((r) => ({ orgId: r.orgId as string, policy: { maxKeyAgeDays: r.maxKeyAgeDays, overlapDays:
r.overlapDays, enabled: r.enabled } }));
},
};

```

FILE: apps/api/src/services/admin/scheduledRotation.service.ts

```

import { isRotationDue, deriveRotationState, type RotationPolicy, type SigningPublicKey } from '@remotedesk/shared';
import { signingKeyRepository } from '../../repositories/signingKeyRepository';
import { rotationPolicyRepository } from '../../repositories/rotationPolicyRepository';
import { deviceAdminAudit } from '../../lib/deviceAdminAudit';

/**
 * Decides + performs scheduled rotation for one org. IDEMPOTENT: it only rotates
 * when the policy says rotation is due, so it's safe to call repeatedly (cron,
 * boot, manual run-now). Overlap is inherent: Pack 27 rotate() mints the new
 * active key THEN retires the old, so signing never has a gap. Retired keys are
 * kept (never deleted), so historical receipts always verify.
 */
export const scheduledRotationService = {
    async runForOrg(orgId: string, policy?: RotationPolicy, now: Date = new Date()): Promise<{ rotated: boolean; newKeyId:
string | null }> {
        const effective = policy ?? (await rotationPolicyRepository.get(orgId));
        const keys: SigningPublicKey[] = await (signingKeyRepository as any).listPublic(orgId);
        const state = deriveRotationState(keys);
        const activeKey = keys.find((k) => k.keyId === state.activeKeyId) ?? null;

        if (!isRotationDue(effective, activeKey, now)) {
            await rotationPolicyRepository.markRun(orgId, now);
            return { rotated: false, newKeyId: null };
        }

        const newKeyId: string = await (signingKeyRepository as any).rotate(orgId);
        await rotationPolicyRepository.markRun(orgId, now);
        await deviceAdminAudit.record({
            orgId, actorUserId: null, deviceId: 'signing-key',
            action: 'device.trust.changed',

```

```

        metadata: { event: 'signing_key_rotated', mode: 'scheduled', newKeyId, ageDays: String(activeKey ?
Math.floor((now.getTime() - Date.parse(activeKey.createdAt)) / 86_400_000) : -1) },
    });
    return { rotated: true, newKeyId };
},

async runAll(now: Date = new Date()): Promise<Array<{ orgId: string; rotated: boolean; newKeyId: string | null }>> {
    const orgs = await rotationPolicyRepository.allEnabled();
    const out: Array<{ orgId: string; rotated: boolean; newKeyId: string | null }> = [];
    for (const { orgId, policy } of orgs) {
        try { out.push({ orgId, ...(await this.runForOrg(orgId, policy, now)) }); }
        catch { out.push({ orgId, rotated: false, newKeyId: null }); } // one org's failure never blocks the rest
    }
    return out;
},
};

```

FILE: apps/api/src/jobs/rotationRunner.ts

```

import { scheduledRotationService } from '../services/admin/scheduledRotation.service';

/**
 * Lightweight in-process scheduler. Calls runAll on an interval. In production
 * prefer an external cron / job queue hitting POST /run-now; this is a safe
 * default. Idempotent runs mean overlapping ticks are harmless.
 */
let timer: ReturnType<typeof setInterval> | null = null;

export function startRotationRunner(intervalMs = 6 * 60 * 60 * 1000): void {
    if (timer) return;
    void safeRun();
    timer = setInterval(() => void safeRun(), intervalMs);
}

export function stopRotationRunner(): void {
    if (timer) clearInterval(timer);
    timer = null;
}

async function safeRun(): Promise<void> {
    try {
        const summary = await scheduledRotationService.runAll();
        const rotated = summary.filter((s) => s.rotated).length;
        if (rotated > 0) console.info('[rotation] scheduled rotation complete', { orgsRotated: rotated });
    } catch (err) {
        console.warn('[rotation] runner error', { reason: err instanceof Error ? err.message : 'unknown' });
    }
}

```

Wire (REVIEW_REQUIRED) in server bootstrap: startRotationRunner() after app + db are ready. Or omit and drive rotation purely via POST /run-now from an external scheduler.

Part D — API: routes

FILE: apps/api/src/routes/deviceAdmin.routes.ts [MODIFY]

```
// ...existing imports...
import { rotationPolicyRepository } from '../repositories/rotationPolicyRepository';
import { scheduledRotationService } from '../services/admin/scheduledRotation.service';
import { requireDeviceAdmin } from '../middleware/requireDeviceAdmin.middleware';
import { z } from 'zod';

const rotationPolicySchema = z.object({
  maxKeyAgeDays: z.number().int().min(1).max(3650).optional(),
  overlapDays: z.number().int().min(0).max(365).optional(),
  enabled: z.boolean().optional(),
});

// ---- ADD: get/set rotation policy (admin) + manual run-now (admin, idempotent) ----
// .get('/receipts/rotation-policy', requireAuth, orgContext, requireDeviceAdmin, h(async (req, res) => {
//   res.json(await rotationPolicyRepository.get((req as OrgRequest).orgId));
// })))
// .put('/receipts/rotation-policy', requireAuth, orgContext, requireDeviceAdmin, h(async (req, res) => {
//   const input = rotationPolicySchema.parse(req.body);
//   res.json(await rotationPolicyRepository.set((req as OrgRequest).orgId, input));
// })))
// .post('/receipts/rotation-policy/run-now', requireAuth, orgContext, requireDeviceAdmin, h(async (req, res) => {
//   res.json(await scheduledRotationService.runForOrg((req as OrgRequest).orgId));
// })))
```

Part E — Web: rotation policy panel

FILE: apps/web/app/dashboard/receipts/RotationPolicyPanel.tsx

```
'use client';

import React, { useEffect, useState } from 'react';
import type { RotationPolicy } from '@remotedesk/shared';
import { api } from '../../../../src/api/apiClient';

type PolicyWithRun = RotationPolicy & { lastRunAt: string | null };

/**
 * Admin panel to configure scheduled key rotation: enable, max key age, overlap
 * window, and a manual rotate-now trigger. No key material shown.
 */
export function RotationPolicyPanel({ canManage }: { canManage: boolean }) {
  const [policy, setPolicy] = useState<PolicyWithRun | null>(null);
  const [busy, setBusy] = useState(false);
  const [msg, setMsg] = useState<string | null>(null);

  const load = async () => setPolicy(await api.get<PolicyWithRun>('/devices/receipts/rotation-policy'));
  useEffect(() => { void load(); }, []);

  const save = async () => {
    if (!policy) return;
    setBusy(true); setMsg(null);
    try {
      await api.put('/devices/receipts/rotation-policy', { maxKeyAgeDays: policy.maxKeyAgeDays, overlapDays:
policy.overlapDays, enabled: policy.enabled });
      setMsg('Saved.');
```

```

    } finally { setBusy(false); }
  };

  const runNow = async () => {
    setBusy(true); setMsg(null);
    try {
      const r = await api.post<{ rotated: boolean; newKeyId: string | null }>('/devices/receipts/rotation-policy/run-now', {});
      setMsg(r.rotated ? `Rotated. New key: ${r.newKeyId}` : 'Not due yet — no rotation needed.');
```

await load();

```

    } finally { setBusy(false); }
  };

  if (!policy) return <section className="rotation-policy-panel"><p>Loading rotation policy...</p></section>;

  return (
    <section className="rotation-policy-panel">
      <h2>Automatic key rotation</h2>
      <label>
        <input type="checkbox" disabled={!canManage} checked={policy.enabled} onChange={(e) => setPolicy({ ...policy, enabled: e.target.checked })} />
        Enable scheduled rotation
      </label>
      <div className="row">
        <label>Rotate after (days)
          <input type="number" min={1} max={3650} disabled={!canManage} value={policy.maxKeyAgeDays} onChange={(e) => setPolicy({ ...policy, maxKeyAgeDays: Number(e.target.value) })} />
        </label>
        <label>Overlap window (days)
          <input type="number" min={0} max={365} disabled={!canManage} value={policy.overlapDays} onChange={(e) => setPolicy({ ...policy, overlapDays: Number(e.target.value) })} />
        </label>
      </div>
      <p className="muted">Last run: {policy.lastRunAt ? new Date(policy.lastRunAt).toLocaleString() : 'never'}</p>
      {canManage && (
        <div className="row">
          <button type="button" disabled={busy} onClick={save}>Save policy</button>
          <button type="button" disabled={busy} onClick={runNow}>Rotate now</button>
        </div>
      )}
      {msg && <p className="muted">{msg}</p>}
      <p className="muted">A new key is created before the old one retires, so signing never stops. Retired keys keep verifying old receipts.</p>
    </section>
  );
}

```

FILE: apps/web/app/dashboard/receipts/page.tsx [MODIFY]

```

// ...existing imports...
import { RotationPolicyPanel } from './RotationPolicyPanel';
// const { canMutate } = useDeviceAdminPermissions(role); // Pack 24

// ---- ADD near the SigningKeyPanel (Pack 27) ----
// <RotationPolicyPanel canManage={canMutate} />

```

Part F — Tests

FILE: tests/admin/rotationPolicy.test.ts

```
import { describe, it, expect } from 'vitest';
import { normalizeRotationPolicy, keyAgeDays, isRotationDue, isWithinOverlap, DEFAULT_ROTATION_POLICY, type
SigningPublicKey } from '@remotedesk/shared';

function key(createdAt: string, retiredAt: string | null = null): SigningPublicKey {
  return { keyId: 'k1', publicKey: 'p', alg: 'ed25519', createdAt, retiredAt };
}

const NOW = new Date('2026-06-15T00:00:00.000Z');

describe('rotationPolicy', () => {
  it('normalizes + clamps untrusted input', () => {
    expect(normalizeRotationPolicy({ maxKeyAgeDays: 0, overlapDays: 9999, enabled: true })).toEqual({ maxKeyAgeDays: 1,
overlapDays: 365, enabled: true });
    expect(normalizeRotationPolicy({})).toEqual(DEFAULT_ROTATION_POLICY);
  });

  it('computes key age in days', () => {
    expect(keyAgeDays(key('2026-05-16T00:00:00.000Z'), NOW)).toBe(30);
  });

  it('rotation not due when disabled', () => {
    expect(isRotationDue({ maxKeyAgeDays: 1, overlapDays: 0, enabled: false }, key('2020-01-01T00:00:00.000Z'),
NOW)).toBe(false);
  });

  it('rotation due when the active key is older than maxKeyAgeDays', () => {
    const policy = { maxKeyAgeDays: 30, overlapDays: 7, enabled: true };
    expect(isRotationDue(policy, key('2026-05-16T00:00:00.000Z'), NOW)).toBe(true); // 30d
    expect(isRotationDue(policy, key('2026-05-20T00:00:00.000Z'), NOW)).toBe(false); // 26d
  });

  it('rotation due (fail-safe) when there is no active key', () => {
    expect(isRotationDue({ maxKeyAgeDays: 90, overlapDays: 7, enabled: true }, null, NOW)).toBe(true);
  });

  it('overlap window includes a freshly retired key, excludes an old one', () => {
    const policy = { maxKeyAgeDays: 90, overlapDays: 7, enabled: true };
    expect(isWithinOverlap(policy, key('x', '2026-06-10T00:00:00.000Z'), NOW)).toBe(true); // retired 5d ago
    expect(isWithinOverlap(policy, key('x', '2026-06-01T00:00:00.000Z'), NOW)).toBe(false); // retired 14d ago
    expect(isWithinOverlap(policy, key('x', null), NOW)).toBe(false); // not retired
  });
});
```

FILE: tests/admin/scheduledRotation.service.test.ts

```
import { describe, it, expect, vi, beforeEach } from 'vitest';

const listPublic = vi.fn();
const rotate = vi.fn();
const getPolicy = vi.fn();
```

```

const markRun = vi.fn().mockResolvedValue(undefined);
const auditRecord = vi.fn().mockResolvedValue(undefined);

vi.mock('../..../apps/api/src/repositories/signingKeyRepository', () => ({
  signingKeyRepository: { listPublic: (...a: unknown[]) => listPublic(...a), rotate: (...a: unknown[]) => rotate(...a) },
}));
vi.mock('../..../apps/api/src/repositories/rotationPolicyRepository', () => ({
  rotationPolicyRepository: { get: (...a: unknown[]) => getPolicy(...a), markRun: (...a: unknown[]) => markRun(...a) },
}));
vi.mock('../..../apps/api/src/lib/deviceAdminAudit', () => ({
  deviceAdminAudit: { record: (...a: unknown[]) => auditRecord(...a) },
}));

import { scheduledRotationService } from '../..../apps/api/src/services/admin/scheduledRotation.service';

function key(createdAt: string, retiredAt: string | null = null) {
  return { keyId: 'k1', publicKey: 'p', alg: 'ed25519', createdAt, retiredAt };
}
const NOW = new Date('2026-06-15T00:00:00.000Z');

describe('scheduledRotationService.runForOrg', () => {
  beforeEach(() => { listPublic.mockReset(); rotate.mockReset(); auditRecord.mockClear(); markRun.mockClear(); });

  it('does not rotate when not due', async () => {
    listPublic.mockResolvedValue([key('2026-06-10T00:00:00.000Z')]); // 5d old
    const r = await scheduledRotationService.runForOrg('org', { maxKeyAgeDays: 90, overlapDays: 7, enabled: true }, NOW);
    expect(r.rotated).toBe(false);
    expect(rotate).not.toHaveBeenCalled();
    expect(markRun).toHaveBeenCalled();
  });

  it('rotates when the active key is too old + audits', async () => {
    listPublic.mockResolvedValue([key('2026-01-01T00:00:00.000Z')]);
    rotate.mockResolvedValue('k2');
    const r = await scheduledRotationService.runForOrg('org', { maxKeyAgeDays: 90, overlapDays: 7, enabled: true }, NOW);
    expect(r.rotated).toBe(true);
    expect(r.newKeyId).toBe('k2');
    expect(rotate).toHaveBeenCalled();
    expect(auditRecord.mock.calls[0][0].metadata.mode).toBe('scheduled');
  });

  it('mints a key when none exists (fail-safe due)', async () => {
    listPublic.mockResolvedValue([]);
    rotate.mockResolvedValue('k1');
    const r = await scheduledRotationService.runForOrg('org', { maxKeyAgeDays: 90, overlapDays: 7, enabled: true }, NOW);
    expect(r.rotated).toBe(true);
  });

  it('does nothing when disabled', async () => {
    listPublic.mockResolvedValue([key('2020-01-01T00:00:00.000Z')]);
    const r = await scheduledRotationService.runForOrg('org', { maxKeyAgeDays: 1, overlapDays: 0, enabled: false }, NOW);
    expect(r.rotated).toBe(false);
    expect(rotate).not.toHaveBeenCalled();
  });

  it('is idempotent: a second immediate run does not double-rotate', async () => {
    listPublic.mockResolvedValueOnce([key('2026-01-01T00:00:00.000Z')]).mockResolvedValueOnce([key('2026-06-

```

```

15T00:00:00.000Z' ]]);
    rotate.mockResolvedValue('k2');
    const first = await scheduledRotationService.runForOrg('org', { maxKeyAgeDays: 90, overlapDays: 7, enabled: true },
NOW);
    const second = await scheduledRotationService.runForOrg('org', { maxKeyAgeDays: 90, overlapDays: 7, enabled: true },
NOW);
    expect(first.rotated).toBe(true);
    expect(second.rotated).toBe(false);
    expect(rotate).toHaveBeenCalledTimes(1);
  });
});

```

Part G — Reports

FILE: [generated-pack30-scheduled-rotation-merge-summary.md](#)

Pack 30 — Scheduled Key Rotation + Overlap Windows · Merge Summary

What this slice does

Makes Pack 27's manual rotation automatic + safe:

1. RotationPolicy (maxKeyAgeDays + overlapDays + enabled), normalized/clamped.
2. Pure evaluator: isRotationDue (age-based, fail-safe when no key) + isWithinOverlap.
3. scheduledRotationService.runForOrg/runAll — IDEMPOTENT: rotates only when due.
4. rotationRunner — in-process interval scheduler (or drive via POST run-now from external cron).
5. Admin web panel: enable, set age + overlap, rotate-now, last-run.

Overlap guarantee

Pack 27 rotate() mints the NEW active key THEN retires the old one — so there is NEVER a moment without a valid signing key. Retired keys are never deleted, so historical receipts always verify; isWithinOverlap just flags the grace window.

Files created (8) / modified (4)

New: shared rotationPolicy; migration; rotationPolicyRepository; scheduledRotation.service; rotationRunner; web RotationPolicyPanel; 2 tests.

Modified: shared index; schema (KeyRotationPolicy); deviceAdmin.routes (get/set/run-now); receipts page.

Files intentionally NOT touched

- apps/desktop/** — Pack 28 pinning already handles new keyIds on rotation.
- kms/** + receiptSigner — envelope crypto + signing-with-active-key unchanged.

Remaining manual wiring risks

1. Call startRotationRunner() in server bootstrap, OR wire an external cron to POST run-now.
2. Decide org-level vs workspace-global policy (model supports orgId null = global; routes use org).
3. requireDeviceAdmin gates policy set + run-now (Pack 24).
4. Multi-instance: prefer external cron / a lock (idempotency makes it safe but noisy).

Verify

- pnpm --filter @remotedesk/shared build && pnpm -r test
- Manual: enable rotation, low maxKeyAgeDays in dev, run-now -> rotates; second run-now -> 'not due'; old receipts still verify (Pack 28/29).

FILE: [generated-pack30-scheduled-rotation-code-review.md](#)

Pack 30 — Code Review Notes

Strengths

- Idempotent rotation: due-check gates the actual rotate, so cron + boot + manual runs can't double-rotate.
- Overlap is structural (mint-before-retire via Pack 27) — no signing gap, ever.
- Retired keys persist, so verification of historical receipts is never affected.
- Policy input clamped/normalized in shared, so API + UI can't persist nonsense.
- runAll isolates per-org failures.
- Pure policy + service (mocks) fully unit-tested incl. idempotency.

Reviewer checklist

- [] startRotationRunner wired once (single instance) or replaced by external cron + run-now.
- [] Multi-instance: add a job lock or external scheduler.
- [] requireDeviceAdmin on policy set + run-now.
- [] Audit event tags mode:'scheduled'.
- [] markRun updates lastRunAt even on no-op runs (it does).

Safety

- No private key handling here; rotation delegates to Pack 27's isolated path.
- New key active before old retires; retired keys verify forever.
- Disabled policy => never rotates.

FILE: generated-pack30-scheduled-rotation-manifest.json

```
{
  "pack": "pack30-scheduled-rotation",
  "title": "Scheduled Key Rotation with Overlap Windows + Rotation Policy",
  "actualFileCount": 12,
  "filesCreated": [
    "packages/shared/src/permissions/rotationPolicy.ts",
    "apps/api/prisma/migrations/0030_rotation_policy/migration.sql",
    "apps/api/src/repositories/rotationPolicyRepository.ts",
    "apps/api/src/services/admin/scheduledRotation.service.ts",
    "apps/api/src/jobs/rotationRunner.ts",
    "apps/web/app/dashboard/receipts/RotationPolicyPanel.tsx",
    "tests/admin/rotationPolicy.test.ts",
    "tests/admin/scheduledRotation.service.test.ts"
  ],
  "filesModified": [
    "packages/shared/src/permissions/index.ts",
    "apps/api/prisma/schema.prisma",
    "apps/api/src/routes/deviceAdmin.routes.ts",
    "apps/web/app/dashboard/receipts/page.tsx"
  ],
  "filesNotTouched": [
    "apps/desktop/**",
    "apps/api/src/lib/kms/**",
    "apps/api/src/services/admin/receiptSigner.service.ts"
  ],
  "dependenciesRequired": {
    "new": [],
    "existing": ["@remotedesk/shared", "@prisma/client", "express", "zod", "next", "react", "vitest"],
    "assumedFromProject": [
      "Pack 27 signingKeyRepository.rotate + listPublic",
      "Pack 25 deviceAdminAudit",
      "Pack 24 requireDeviceAdmin",
      "a scheduler (in-process runner provided) or external cron hitting run-now"
    ]
  }
}
```

```

    },
    "knownLimitations": [
      "In-process runner is single-instance friendly; multi-instance needs a lock or external cron (idempotency keeps it safe).",
      "Policy is per-org via routes; workspace-global supported by the model but not surfaced in UI.",
      "Overlap window is advisory; retired keys verify regardless, by design.",
      "No re-sign of old receipts under a new key (not needed — old keys verify).",
    ],
    "remainingManualWiringRisks": [
      "Start the runner once or wire external cron to run-now.",
      "Add a job lock for multi-instance deployments.",
      "Keep requireDeviceAdmin on policy mutation + run-now."
    ],
    "estimatedCompletionAfterCarefulMerge": "89-93%"
  }
}

```

Pack 30 done: 12 files (8 created, 4 modified). Key rotation is now automatic and gap-free. A rotation policy (max key age + overlap window + enable switch) drives an idempotent scheduled runner that rotates only when the active key is older than the configured age, and a just-retired key stays flagged "in overlap" through a grace window. Admins configure it all from the receipts page with a manual rotate-now button.

The clean bit: the overlap guarantee is structural, not timing-luck. Pack 27's rotate() mints the new active key before retiring the old one, so there's never an instant without a valid signing key, and because retired keys are never deleted, every historical receipt keeps verifying through Pack 28/29 regardless of how many rotations have happened. The runner is idempotent: the due-check gates the actual rotate, so cron ticks, a boot kick, and a manual run-now can all fire without ever double-rotating (a test proves a second immediate run is a no-op). Policy input is clamped in shared, so neither the API nor the UI can persist nonsense.

That completes the receipt key lifecycle: protected (27) -> host-verified (28) -> publicly auditable (29) -> automatically rotated with zero downtime (30). The whole trust/consent/receipt system is now production-shaped end to end.

Deploy note: the in-process runner is single-instance friendly; for multiple API instances drive rotation from external cron hitting run-now (idempotency keeps it safe either way). The trust/receipt feature area is essentially done, so the next move is probably a fresh feature. Want to pick a new direction, or keep hardening this one? Bolo.